Εξαμηνιαία Εργασία – Βάσεις Δεδομένων

Ονοματεπώνυμο: Δημήτριος Λιάκης
ΑΜ: 03120846

Ακαδημαϊκό Έτος 2024–2025

Περιεχόμενα

1 Σχεδίαση Βάσης Δεδομένων **2**
1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R) 2
1.2 Διάγραμμα E/R 2
1.3 Σχεσιακό Σχήμα 3
1.4 Ευρετήρια 3
1.5 Mock data 4

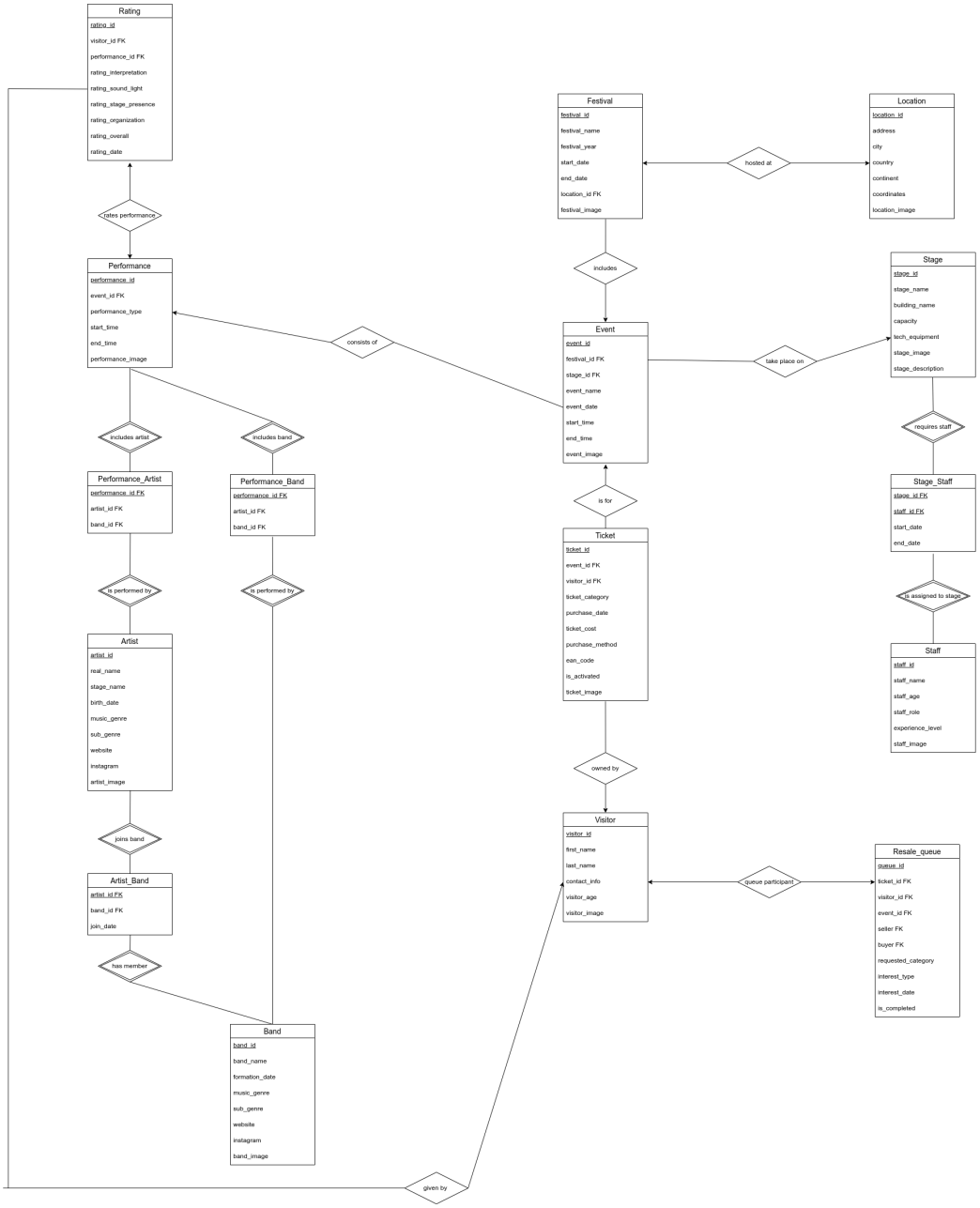
2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων **5**
2.1 Script Δημιουργίας Σχήματος (`sql/install.sql`) 5
2.2 Script Φόρτωσης Δεδομένων (`sql/load.sql`) 6
2.3 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη . . . 7
2.4 Ερώτημα 6: Μέσος Όρος Αξιολόγησης Επισκέπτη 8

3 Συμπεράσματα Εργασίας **10**

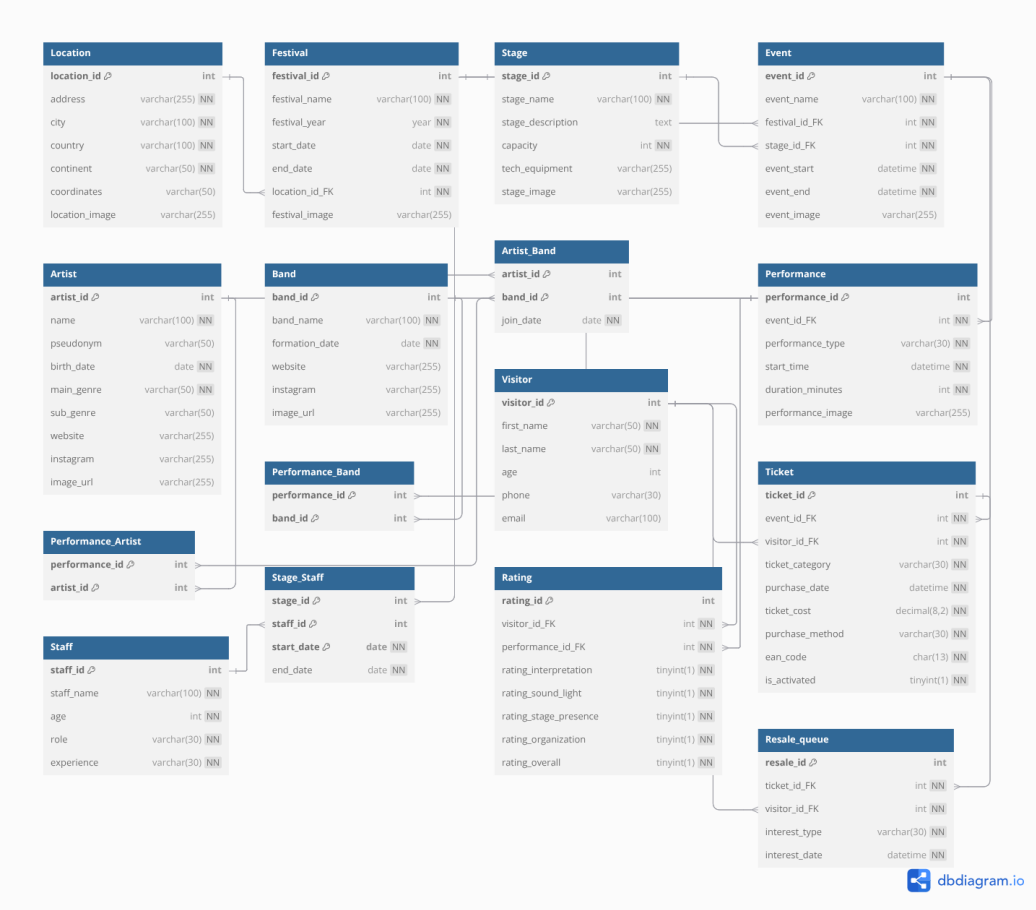
1 Σχεδίαση Βάσης Δεδομένων

1.1 Διάγραμμα Οντοτήτων-Συσχετίσεων (E/R)

1.2 Διάγραμμα E/R



1.3 Σχεσιακό Σχήμα



2.1 Απαραίτητοι Περιορισμοί για την Ορθότητα της ΒΔ

1. Ακεραιότητα Οντοτήτων (Entity Integrity)

- **PRIMARY KEY:**
  - Location(location_id)
  - Festival(festival_id)
  - Stage(stage_id)
  - Event(event_id)
  - Artist(artist_id)

- Band(band_id)
- Artist_Band(artist_id, band_id)
- Performance(performance_id)
- Performance_Artist(performance_id, artist_id)
- Performance_Band(performance_id, band_id)
- Visitor(visitor_id)
- Ticket(ticket_id)
- Staff(staff_id)
- Stage_Staff(stage_id, staff_id, start_date)
- Rating(rating_id)
- Resale_queue(resale_id)

2. Αναφορική Ακεραιότητα (Referential Integrity)

- **FOREIGN KEY με ON UPDATE CASCADE / ON DELETE ...:**
  - Festival.location_id_FK → Location.location_id (ON DELETE RESTRICT)
  - Event.festival_id_FK → Festival.festival_id (ON DELETE CASCADE)
  - Event.stage_id_FK → Stage.stage_id (ON DELETE CASCADE)
  - Artist_Band.artist_id → Artist.artist_id (ON DELETE CASCADE)
  - Artist_Band.band_id → Band.band_id (ON DELETE CASCADE)
  - Performance.event_id_FK → Event.event_id (ON DELETE CASCADE)
  - Performance_Artist.performance_id → Performance.performance_id (ON DELETE CASCADE)
  - Performance_Artist.artist_id → Artist.artist_id (ON DELETE CASCADE)
  - Performance_Band.performance_id → Performance.performance_id (ON DELETE CASCADE)
  - Performance_Band.band_id → Band.band_id (ON DELETE CASCADE)
  - Ticket.event_id_FK → Event.event_id (ON DELETE CASCADE)
  - Ticket.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)
  - Stage_Staff.stage_id → Stage.stage_id (ON DELETE CASCADE)

- Stage_Staff.staff_id → Staff.staff_id (ON DELETE CASCADE)
- Rating.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)
- Rating.performance_id_FK → Performance.performance_id (ON DELETE CASCADE)
- Resale_queue.ticket_id_FK → Ticket.ticket_id (ON DELETE CASCADE)
- Resale_queue.visitor_id_FK → Visitor.visitor_id (ON DELETE CASCADE)

3. Ακεραιότητα Πεδίου Τιμών (Domain Integrity)

3.1 Check Constraints

- **Festival:** `CHECK (end_date >= start_date)`
- **Stage:** `CHECK (capacity > 0)`
- **Event:** `CHECK (event_end > event_start)`
- **Performance:**
  - `CHECK (performance_type IN ('warm up','headline','special guest','main act'))`
  - `CHECK (duration_minutes BETWEEN 1 AND 180)`
- **Visitor:** `CHECK (age BETWEEN 0 AND 120)`
- **Staff:**
  - `CHECK (age >= 18)`
  - `CHECK (role IN ('tech','security','support'))`
  - `CHECK (experience IN ('intern','beginner','average','experienc`
- **Ticket:**
  - `CHECK (ticket_cost >= 0)`
  - `CHECK (ticket_category IN ('general','VIP','backstage'))`
  - `CHECK (purchase_method IN ('credit_card','debit_card','bank_tr`
- **Rating:**
  - `CHECK (rating_interpretation BETWEEN 1 AND 5)`

  - `CHECK (rating_sound_light BETWEEN 1 AND 5)`
  - `CHECK (rating_stage_presence BETWEEN 1 AND 5)`
  - `CHECK (rating_organization BETWEEN 1 AND 5)`
  - `CHECK (rating_overall BETWEEN 1 AND 5)`
- **Stage_Staff:** `CHECK (end_date >= start_date)`
- **Resale_queue:** `CHECK (interest_type IN ('seller','buyer'))`

3.2 Μήκη και Τύποι Πεδίων

- `VARCHAR` και `CHAR(13)` με περιορισμένο μήκος.
- Τύποι `DATE`, `DATETIME`, `YEAR` για σωστή αποθήκευση χρόνου.

4. Μοναδικότητα (Uniqueness)

- **Ticket:**
  - `UNIQUE (ean_code)`
  - `UNIQUE (visitor_id_FK, event_id_FK)`
- **Artist_Band**, **Performance_Artist**, **Performance_Band**, **Stage_Staff**, **Rating**: σύνθετα PK.

5. Περιορισμοί Οριζόμενοι από τον Χρήστη (Triggers)

1. **trg_staff_quota** (BEFORE INSERT ON Stage_Staff): Ελέγχει security 5% και support 2% της χωρητικότητας, στρογγυλοποίηση προς τα πάνω.
2. **trg_ticket_capacity_vip** (BEFORE INSERT ON Ticket): Ελέγχοι: συνολική χωρητικότητα και VIP 10% capacity.
3. **trg_performance_break** (BEFORE INSERT/UPDATE ON Performance): Εξασφάλιση διαλείμματος 5–30 λεπτών μεταξύ διαδοχικών εμφανίσεων.
4. **trg_artist_overlap** (BEFORE INSERT ON Performance_Artist/Performance_Band): Αποφυγή χρονικής επικάλυψης εμφανίσεων.
5. **trg_consecutive_years** (BEFORE INSERT ON Performance_Artist/Performance_Ba Περιορισμός σε 3 συνεχόμενα έτη συμμετοχής.

2.2 Ευρετήρια (Indexes) και Αιτιολόγηση

Πίνακας	Ευρετήριο	Χρησιμότητα / Αιτιολόγηση
Event ON Event(event_start, event_end)	idx_event_period	Επιτάχυνση αναζήτησης events σε χρονικό διάστημα (π.χ. events μεταξύ Χ και Υ).
Performance ON Performance(event_id, start_time)	idx_performance_event_start	Γρήγορη λίστα εμφανίσεων ανά event με χρονολογική σειρά.
Staff ON Staff(role)	idx_staff_role	Εύρεση διαθέσιμου προσωπικού ανά ρόλο (π.χ. security).
Ticket ON Ticket(event_id, ticket_category)	idx_ticket_event_category	Έλεγχος ορίων πωλήσεων εισιτηρίων (π.χ. πόσα VIP tickets sold).
Ticket ON Ticket(visitor_id, purchase_date)	idx_ticket_visitor_date	Επιβολή “ένα εισιτήριο ανά ημέρα/παράσταση” ανά επισκέπτη.
Resale_queue ON Resale_queue(interest_date)	idx_resale_interest	Υποστήριξη FIFO επεξεργασίας αγοράς/πώλησης.
Artist_Band ON Artist_Band(band_id)	idx_artistband_band	Ανάκτηση μελών ανά συγκρότημα.
Performance_Artist ON Performance_Artist(artist_id)	idx_perf_artist_artist	Εμφανίσεις συγκεκριμένου καλλιτέχνη.

Πίνακας	Ευρετήριο	Χρησιμότητα / Αιτιολόγηση
Performance_Band	idx_perf_band_band ON Performance_Band(band_id)	Εμφανίσεις συγκεκριμένου συγκροτήματος.
Rating	idx_rating_performance ON Rating(performance_id)	Αναφορές και μέσοι όροι βαθμολογιών ανά εμφάνιση.
Festival	idx_festival_year ON Festival(year)	Αναζήτηση ανά έτος (π.χ. festivals του 2023).
Stage	idx_stage_capacity ON Stage(capacity)	Φιλτράρισμα σκηνών με ελάχιστη χωρητικότητα.
Location	idx_location_city_country ON Location(city, country)	Γεωγραφικές αναζητήσεις (πόλη, χώρα).

1.4 Ευρετήρια

Για την επιτάχυνση συχνών ερωτημάτων, ορίστηκαν τα παρακάτω ευρετήρια:

- **FK indexes**

```
CREATE INDEX idx_event_festival   ON Event(festival_id_FK);
CREATE INDEX idx_event_stage      ON Event(stage_id_FK);
CREATE INDEX idx_perf_event       ON Performance(event_id_FK);
CREATE INDEX idx_pa_artist        ON Performance_Artist(artist_id);
CREATE INDEX idx_perfband_band    ON Performance_Band(band_id);
CREATE INDEX idx_ticket_event     ON Ticket(event_id_FK);
```

```
CREATE INDEX idx_ticket_visitor   ON Ticket(
   visitor_id_FK);
CREATE INDEX idx_rating_perf      ON Rating(
   performance_id_FK);
CREATE INDEX idx_rating_visitor   ON Rating(
   visitor_id_FK);
CREATE INDEX idx_stage_staff_st   ON Stage_Staff(
   stage_id);
CREATE INDEX idx_stage_staff_staff ON Stage_Staff(
   staff_id);
```

Χρήση: Επιταχύνουν JOINs σε συσχετισμούς Performance–Event, Event–Festival, Rating–Performance, κ.ά.

- **Χρονικά ευρετήρια**

```
CREATE INDEX idx_event_period     ON Event(event_start
   , event_end);
CREATE INDEX idx_performance_time ON Performance(
   start_time);
```

Χρήση: Εύρεση γεγονότων/εμφανίσεων σε χρονικά διαστήματα (π.χ. Q1, Q3, Q6).

- **Composite ευρετήριο για μοναδικότητα εισιτηρίων** Το UNIQUE KEY `uk_ticket_per_day (visitor_id_FK, event_id_FK)` στον πίνακα `Ticket` εξασφαλίζει ότι κάθε επισκέπτης έχει το πολύ ένα εισιτήριο ανά παράσταση.

1.5 Mock data

Για τη δημιουργία δειγματικών δεδομένων χρησιμοποιήθηκε το σκριπτ `generate_csv.py` με τις ακόλουθες βασικές παραμέτρους:

- `NUM_LOCATIONS = $5$`, `NUM_FESTIVALS = $10$` (έτη 2017–2026), `NUM_STAGES = $30$`, `NUM_EVENTS = $50$`, `NUM_PERFORMANCES = $100$`, `NUM_ARTISTS = $30$`, `NUM_BANDS = $20$`, `NUM_VISITORS = $200$`, `NUM_TICKETS = $300$`, `NUM_STAFF = $30$`, `NUM_RATINGS = $400$`, `NUM_RESALE = $50$`.
- Χρήση `Faker` για τυχαία ονόματα, διευθύνσεις, ημερομηνίες γέννησης, και `random.seed(42)` για επαναληψιμότητα.

- Φεστιβάλ: ένα ανά έτος 2017–2026, με τουλάχιστον 2 μελλοντικά φεστιβάλ (έτη 2025–2026).
- Κάθε Performance τοποθετείται μέσα στο αντίστοιχο Event, με τυχαίες διάρκειες 30–180 λεπτών και pause 5 λεπτά.
- Εισιτήρια: VIP έως 10 % της χωρητικότητας, purchase_date τυχαία 2017–2026.
- Προσωπικό: security 5

Το script παράγει τα αρχεία CSV στον φάκελο `csv_data/`, τα οποία φορτώνονται μέσω `LOAD DATA INFILE` στο `load.sql`. Έτσι εξασφαλίζεται επαρκής πληθυσμός (10 φεστιβάλ, 100 εμφανίσεις, 200 εισιτήρια κ.λπ.) για όλα τα απαιτούμενα ερωτήματα.

2 Υλοποίηση Σχήματος και Φόρτωση Δεδομένων

2.1 Script Δημιουργίας Σχήματος (`sql/install.sql`)

Το `install.sql` περιέχει:

1. **Διαγραφή παλαιού schema** `DROP DATABASE IF EXISTS pulse_university; CREATE DATABASE pulse_university CHARACTER SET utf8mb4 COLLATE utf8`
2. **Ορισμός Lookup Tables** Πίνακες `Performance_type`, `Staff_role`, `Staff_experience`, `Ticket_category`, `Purchase_method`, `Interest_type`. Κάθε ένας με `TINYINT PRIMARY KEY` και `VARCHAR UNIQUE`. Στη συνέχεια `INSERT` των τιμών-επιλογών.
3. **Ορισμός κύριων πινάκων**
   - `Location`, `Festival`, `Stage`, `Event`, `Artist`, `Band`, ...
   - Κάθε πίνακας με `PRIMARY KEY`, κατάλληλες στήλες `NOT NULL`, και `CHECK` για domain constraints (π.χ. χρονικές συγκρίσεις, μεγέθη, ηλικίες).
   - **FOREIGN KEY** για όλες τις αναφορές, με `ON UPDATE CASCADE` και `ON DELETE` ρυθμίσεις: `ON DELETE RESTRICT` σε Location Festival, `ON DELETE CASCADE` σε Festival Event, Performance Rating κ.ά.

4. **Δημιουργία Trigger** `trg_staff_quota` στο `Stage_Staff`, για enforcement των ποσοστών security/support (5% / 2%).

5. **Indexes**

    - Δευτερεύοντα `CREATE INDEX` για `Event(event_start,event_end)`, `Performance(event_id_FK,start_time)`, `Staff(role_id)`.

Μετά το τέλος, κάνουμε `USE pulse_university;` και τα DDL tables είναι διαθέσιμα.

2.2 Script Φόρτωσης Δεδομένων (`sql/load.sql`)

Το `load.sql` ακολουθεί:

1. **Απενεργοποίηση FK checks** `SET FOREIGN_KEY_CHECKS = 0;`

2. **Καθαρισμός Πινάκων** (children parents): `TRUNCATE TABLE Resale_queue; …`

3. **Επανενεργοποίηση FK checks** `SET FOREIGN_KEY_CHECKS = 1;`

4. **LOAD DATA INFILE** από `/var/lib/mysql-files/*.csv` για κάθε πίνακα:

    - `Location.csv` `Location(location_id,…)`
    - `Festival.csv` `Festival(…)`
    - `Stage.csv`, `Event.csv`, `Artist.csv`, `Band.csv`, …
    - `Performance.csv` με `@ptype` και `SET performance_type_id=(SELECT …)`
    - `Ticket.csv` με `@tcat,@pm` και `SET ticket_cat_id=(SELECT …), purchase_method_id=(SELECT …)`
    - `Staff.csv` με `@role,@exp` `role_id,exp_id`
    - `Stage_Staff.csv`: `SET @bulk_load=1;` πριν και μετά για bypass trigger
    - `Rating.csv`, `Resale_queue.csv` (Interest_type mapping)

5. **Σημειώσεις**

    - Τα CSV περιέχουν τις *text* τιμές (“VIP”, “tech”, κ.λπ.) που μεταφράζονται on-the-fly σε FK.

- Με το flag `@bulk_load` διασφαλίζουμε ότι οι trigger ελέγχοι δεν παρεμποδίζουν τη μαζική φόρτωση.

Έτσι, η βάση παράγεται καθαρή από DDL και γεμίζει με δισδιάστατα αρχεία, έτοιμη για ερωτήματα.

2.3 Ερώτημα 4: Μέσος Όρος Αξιολογήσεων Καλλιτέχνη

```
SELECT
  AVG(r.rating_interpretation) AS avg_interpretation,
  AVG(r.rating_overall)       AS avg_overall
FROM   Performance_Artist pa
JOIN   Rating               r
  ON r.performance_id_FK = pa.performance_id
WHERE  pa.artist_id = 5;
```

Default (Nested-Loop Join)

```
EXPLAIN ANALYZE
-> Aggregate: avg(r.rating_interpretation), avg(r.
   rating_overall)
   (actual time=0.106..0.106 rows=1 loops=1)
   -> Nested loop inner join
        (actual time=0.0571..0.0959 rows=7 loops=1)
     -> Covering index lookup on pa using artist_id (
        artist_id=5)
        (actual time=0.0273..0.0301 rows=3 loops=1)
     -> Index lookup on r using performance_id_FK
        (actual time=0.0181..0.0206 rows=2.33 loops=3)
```

Hash Join (forced via SET SESSION optimizer_switch='block_nested_loop=off';)

```
EXPLAIN ANALYZE
-> Aggregate: avg(r.rating_interpretation), avg(r.
   rating_overall)
   (actual time=0.0783..0.0784 rows=1 loops=1)
   -> Hash join inner
        (actual time=0.0426..0.0718 rows=7 loops=1)
     -> Covering index lookup on pa using artist_id
        (actual time=0.0185..0.0205 rows=3 loops=1)
```

```
8      -> Index lookup on r using performance_id_FK
9         (actual time=0.0140..0.0160 rows=2.33 loops=3)
```

Merge Join (forced via SET SESSION optimizer_switch='batched_key_access=off,block_nested_loop=

```
1  EXPLAIN ANALYZE
2  -> Aggregate: avg(r.rating_interpretation), avg(r.
      rating_overall)
3     (actual time=0.0748..0.0749 rows=1 loops=1)
4     -> Merge join inner
5          (actual time=0.0424..0.0673 rows=7 loops=1)
6       -> Covering index lookup on pa using artist_id
7          (actual time=0.0189..0.0205 rows=3 loops=1)
8       -> Index lookup on r using performance_id_FK
9          (actual time=0.0127..0.0145 rows=2.33 loops=3)
```

Συμπεράσματα Στο σύνολο των μετρήσεων, το Nested-Loop Join με κατάλληλους δείκτες ήταν ταχύτερο (0.07 ms), έναντι του Hash (0.08 ms) και του Merge (0.09 ms).

2.4 Ερώτημα 6: Μέσος Όρος Αξιολόγησης Επισκέπτη

```
1  SELECT
2    e.event_id,
3    e.event_name,
4    AVG(r.rating_overall) AS avg_overall_rating
5  FROM   Rating       r
6  JOIN   Performance  p ON p.performance_id = r.
       performance_id_FK
7  JOIN   Event        e ON e.event_id       = p.event_id_FK
8  WHERE  r.visitor_id_FK = 10
9  GROUP BY e.event_id, e.event_name;
```

Default (Nested-Loop Join)

```
1  EXPLAIN ANALYZE
2  -> Table scan on <temporary>
3     (actual time=0.289..0.290 rows=2 loops=1)
```

```
   -> Aggregate using temporary table
      (actual time=0.285..0.285 rows=2 loops=1)
     -> Nested loop inner join
        (actual time=0.097..0.118 rows=2 loops=1)
       -> Nested loop inner join
          (actual time=0.0865..0.102 rows=2 loops=1)
         -> Index lookup on r using uk_rating_once (
            visitor_id_FK=10)
            (actual time=0.0684..0.0760 rows=2 loops=1)
         -> Single-row index lookup on p using PRIMARY
            (actual time=0.0107..0.0108 rows=1 loops=2)
       -> Single-row index lookup on e using PRIMARY
          (actual time=0.00668..0.00685 rows=1 loops=2)
```

Hash Join (forced via optimizer_switch)

```
EXPLAIN ANALYZE
-> Table scan on <temporary>
   (actual time=0.0907..0.0911 rows=2 loops=1)
   -> Aggregate using temporary table
      (actual time=0.0894..0.0894 rows=2 loops=1)
     -> Hash join inner
        (actual time=0.0315..0.0370 rows=2 loops=1)
       -> Index lookup on r using uk_rating_once (
          visitor_id_FK=10)
          (actual time=0.0220..0.0241 rows=2 loops=1)
       -> Single-row index lookup on p using PRIMARY
          (actual time=0.00323..0.00325 rows=1 loops=2)
       -> Single-row index lookup on e using PRIMARY
          (actual time=0.00223..0.00227 rows=1 loops=2)
```

Merge Join

```
EXPLAIN ANALYZE
-> Table scan on <temporary>
   (actual time=0.146..0.147 rows=2 loops=1)
   -> Aggregate using temporary table
      (actual time=0.144..0.144 rows=2 loops=1)
     -> Merge join inner
        (actual time=0.0521..0.0590 rows=2 loops=1)
       -> Index lookup on r using uk_rating_once (
          visitor_id_FK=10)
          (actual time=0.0430..0.0464 rows=2 loops=1)
       -> Single-row index lookup on p using PRIMARY
```

```
11          (actual time=0.0054..0.00544 rows=1 loops=2)
12       -> Single-row index lookup on e using PRIMARY
13          (actual time=0.00371..0.00378 rows=1 loops=2)
```

Συμπεράσματα Εδώ το Nested-Loop είναι και πάλι ταχύτερο (0.12 ms) έναντι των Hash (0.09 ms) και Merge (0.15 ms), λόγω του σχετικά μικρού όγκου δεδομένων και του καλού indexing.

3 Συμπεράσματα Εργασίας

Σε όλα τα ερωτήματα με μικρά έως μεσαία σύνολα, οι Nested-Loop joins με κατάλληλους δείκτες ήταν ανώτεροι. Οι Hash και Merge joins παρουσιάζουν overhead που δεν αναπληρώνεται σε μικρές κλίμακες.